

1ª Questão (2,0 pontos): Semáforos

Considere o código produtor-consumidor usando semáforos abaixo do livro de referência do curso (Robert Sebesta. *Conceitos de Linguagens de Programação*).

```
1 semaphore fullspots, emptyspots;
2 fullspots.count = 0;
3 emptyspots.count = BUFLen;
4
5 task producer() {
6     while true {
7         -- Produz VALOR --
8         wait(emptyspots);
9         // espera por um espaço
10        DEPOSITA(VALOR);
11        release(fullspots); // aumenta os espaços preenchidos
12    }
13 }
14
15 task consumer() {
16     while true {
17         wait(fullspots); // certifica-se de que não está vazio
18         RECUPERA(VALOR);
19         release(emptyspots); // aumenta os espaços vazios
20         -- Consome VALOR --
21     }
22 }
```

- Quantas tarefas PRODUTORAS podem existir? Justifique.
- Quantas tarefas CONSUMIDORAS podem existir? Justifique.
- Considere que um programa com produtores-consumidores baseado neste código contém um bug e as tarefas produtoras não foram iniciadas. O que irá ocorrer com as tarefas consumidoras?

2ª Questão (2,0 pontos): Corrotinas

Considere o pseudo-código abaixo onde estão descritas as corrotinas `novo_trabalho()` e `imprime_trabalhos()`.

```
1
2 Fila trabalhos;
3 Trabalho t
4
5 corrotina novo_trabalho()
6     t_conv = converte_eps()
7     fila.insere(t_conv)
8     while fila.tamanho > 0
9         resume imprime_trabalho()
10        libera_recursos()
11
12 corrotina imprime_trabalhos()
13     inicializa_servico()
14     while true
15         conecta_imprensa()
16         while fila.tamanho > 0
17             t = fila.primeiro()
18             envia_impressao(t)
19             resume novo_trabalho()
```

Enumere as linhas dos códigos que serão executadas, obedecendo a sequência de execução, caso sejam executadas as invocações abaixo. Considere que `t` e `trabalhos` foram previamente inicializadas e o programa recém-iniciado.

- a) Execução `resume novo_trabalho()`
- b) Execução `resume imprime_trabalhos()`

3ª Questão (2,0 pontos): Seção Crítica

Considere o código da questão anterior, que faz uso da variável compartilhada `trabalhos`. Considere esse código em uma linguagem que utiliza threads ou gorotinas, no local de `corrotina`, e que a fila `trabalhos` não é thread-safe. Ajuste o código com uso de semáforos, monitores ou gorotinas/channels de maneira que o código fique correto (você está livre para escolher qualquer uma das três opções de abstrações). Você deve escrever somente um pseudo-código que não precisa ser executável e nem em uma linguagem existente.

4ª Questão (2,0 pontos): Go e Channels

Para cada um dos códigos Go abaixo, indique a saída do programa quando ele é executado ou o problema que ocorre. Explique a sua resposta baseado no código e no funcionamento de Go.

a) Código A

```
1 func dispara_gorotinas() {
2     fmt.Println("-- inicio --")
3     fim := make(chan bool)
4
5     go func () {
6         fmt.Println(" -- primeiro --")
7         fim <- true
8     }()
9
10    go func () {
11        fmt.Println(" -- segundo --")
12    }()
13
14    <- fim
15    <- fim
16
17 }
18
19 func main() {
20     dispara_gorotinas()
21 }
```

b) Código B

```
1 func dispara_gorotinas() {
2     fmt.Println("-- inicio --")
3     fim := make(chan bool)
4
5     go func () {
6         fmt.Println(" -- primeiro --")
7         fim <- true
8     }()
9
10    go func () {
11        fmt.Println(" -- segundo --")
12        fim <- true
13    }()
14
15
16    <- fim
17    <- fim
18    <- fim
19 }
20
21 func main() {
22     dispara_gorotinas()
23 }
```

c) Código C

```
1 func dispara_gorotinas() {
2     fmt.Println("-- inicio --")
3     fim := make(chan int)
4
5     go func () {
6         fmt.Println(" -- primeiro --")
7         fim <- 2
8     }()
9 }
```

```

9
10     go func () {
11         fmt.Println(" -- segundo --")
12         fim <- (2 * (<- fim))
13     }()
14
15     fmt.Println(<- fim)
16 }
17
18 func main() {
19     dispara_gorotinas()
20 }

```

4ª Questão (2,0 pontos): Go e Channels

O código Python abaixo implementa um padrão em programação concorrente chamado **Lightswitch** (interruptor) para controlar um semáforo que segue o seguinte comportamento:

- A primeira tarefa a usar o **lock** bloqueia o semáforo (passado como parâmetro **semaphore**)
- A última tarefa ativa a usar o **unlock** desbloqueia o semáforo.
- Observe que espera-se que o semáforo **semaphore** seja utilizado para controlar algo usado por outras tarefas e não o **Lightswitch** propriamente.

```

1 class Lightswitch :
2     def __init__(self):
3         self.counter = 0
4         self.mutex = Semaphore (1)
5
6     def lock(self, semaphore ):
7         self.mutex.wait ()
8         self.counter += 1
9         if self.counter == 1:
10             semaphore.wait()
11         self.mutex.signal ()
12
13     def unlock(self, semaphore):
14         self.mutex.wait ()
15         self.counter -= 1
16         if self.counter == 0:
17             semaphore.signal()
18         self.mutex.signal ()

```

Reescreva esse código em Go, usando goroutines e channels. Considere que o código equivalente utilizará **counter** e **mutex** como variáveis globais.

Importante

- O seu código não precisa ser funcional e nem aderir completamente à linguagem Go. Pense como um pseudo-código que usa o conceito de channels, como Go.
- Observe que o semáforo também será implementado como channel e deve possuir funções **wait_semaforo** e **signal_semaforo** para manipulá-lo, em substituição ao uso de **wait()** e **signal** nas linhas 10 e 17, respectivamente.